GAZİ UNIVERSITY

FACULTY OF ENGINEERING
DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE104-COMPUTER PROGRAMMING I

WEEK 11: METHODS

Introduction

- In object-oriented programming languages, functions are often called "methods".
- The concepts to be expressed as method and function will be used interchangeably.
- Every running C# program has a Main function. The compiler and runtime program starts to run from this function.
- For complicated programs, we can’t write all of the codes inside this function.
- Methods are Subprograms that are designed to perform a specific task for sole purpose of reusing that subprogram multiple times.
- Methods are can not be used stand-alone and do useful works for the main program where they were called.

Introduction

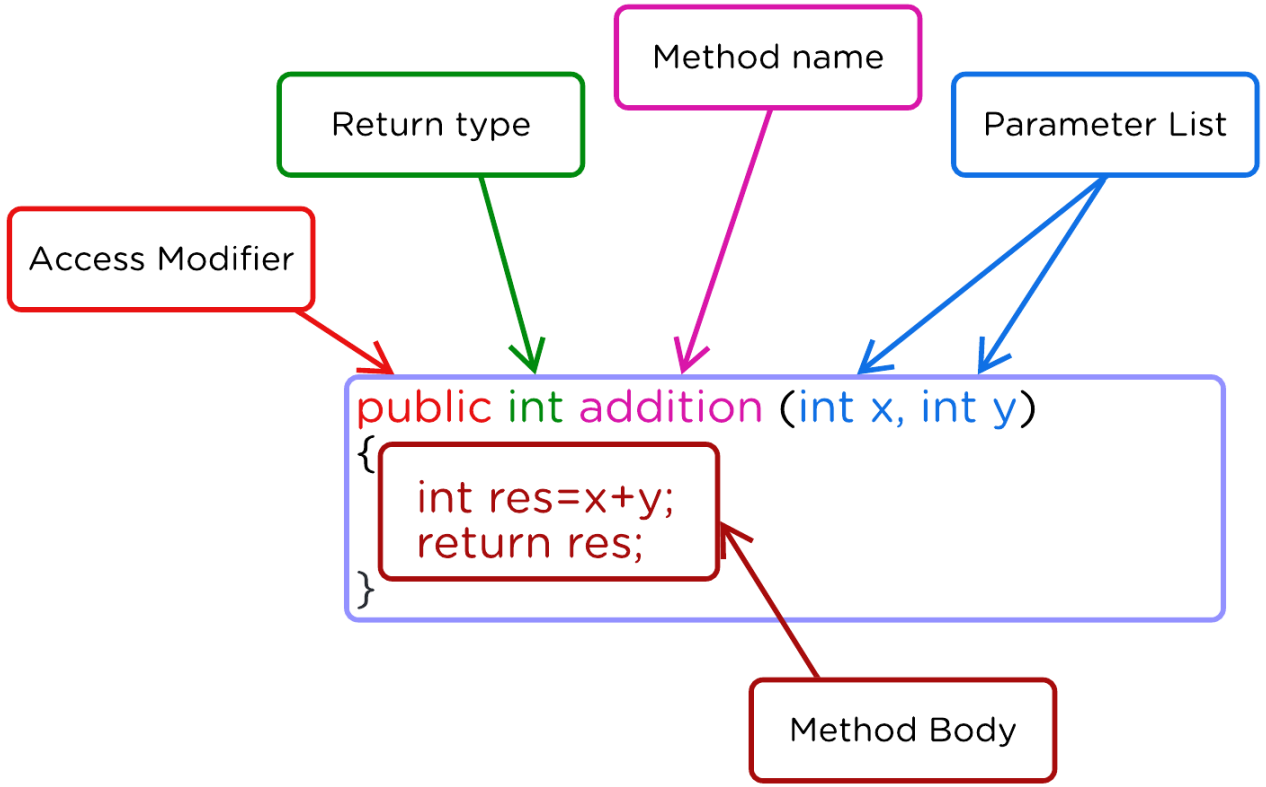
For the method structure;

- The information received from the invoking method is called parameter.
- The information returned to the invoking method is called return value.

Methods Structure

- All methods declared in C# must be a part of a class.
- Methods that are not members of a class cannot be declared.
- The declaration used to define methods is as follows:

```
Access modifiers        <return type> method_name(parameters)
  {
     method body;
  }
```



Methods Features

- Access modifiers (public, private..) set where the method can be accessed. If not specified, it is considered "private".
- Private methods can only be used in the class they are defined in.
- The return value is a type of data that is sent to the invoking method.
- Parameters are the information send from the invoking method that the target method needs while running.

Method Statement

- If a method is to be used in C#, an object of the class is defined and the method is called with the '.' operator.

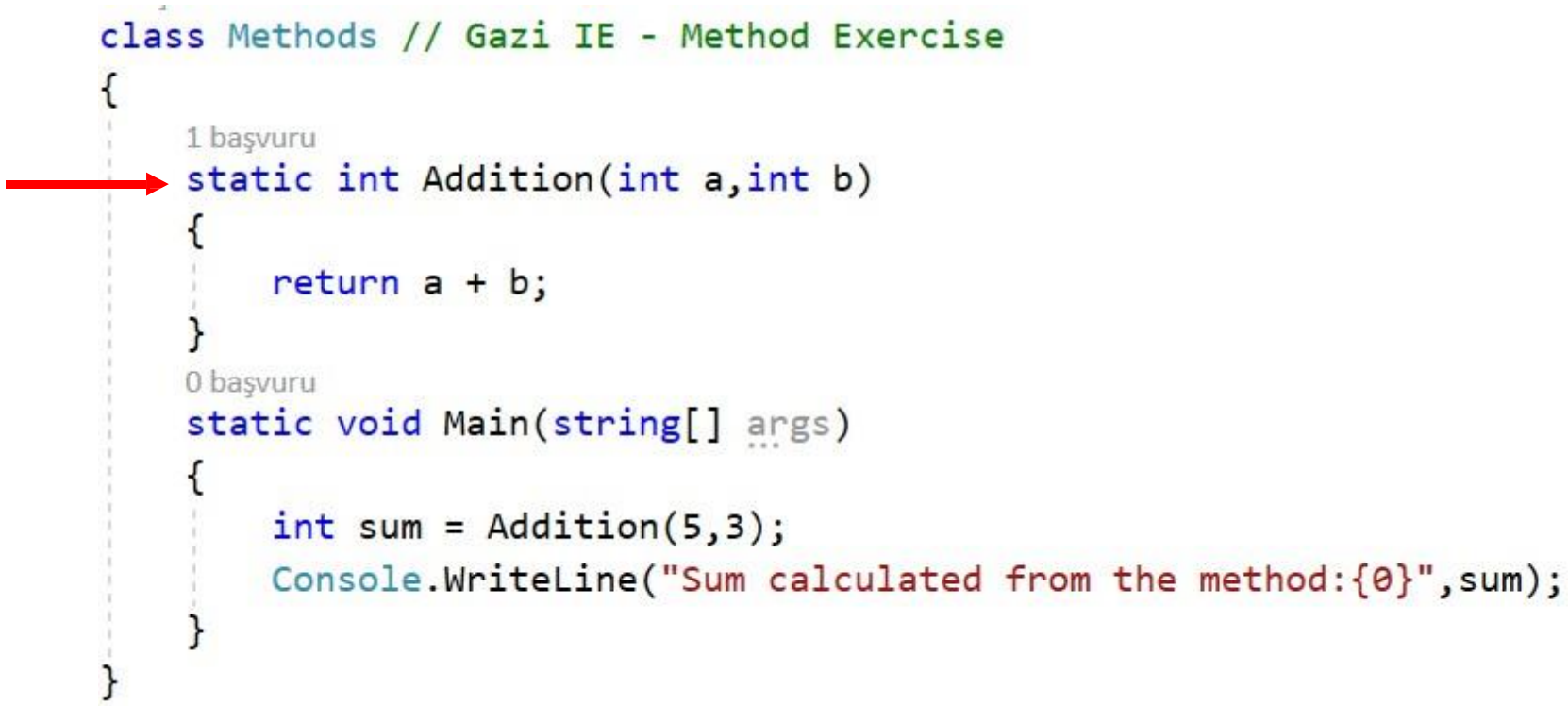
```
class Methods // Gazi IE - Method Exercise
{
    1 başvuru
    int Addition(int a,int b)
    {
        return a + b;
    }
    0 başvuru
    static void Main(string[] args)
    {
        Methods obj = new Methods();
        int sum = obj.Addition(5,3);
        Console.WriteLine("Sum calculated from the method:{0}",sum);

    }
}
```

Expected Output➔

```
Sum calculated from the method:8
```

Method Statement

- Static methods belong to the class rather than to an object. Therefore, they can be called without creating an object of the class if the call is within the same class. Usually, a static method is accessed by writing the class name followed by the method name.



Method Statement

- In all programs, the Main method runs first.
- If the method is not created in the current class, but in another class, the class name must be written first in order to use that method.

There is two way to create methods:

- ❑ Static methods → With a static keyword, no need to create an object.
- ❑ Non-static methods → Must create an object.

```
1  using System;
2
3  class Araba
4  {
5      public static bool Calistir(int BataryaSeviyesi, int BenzinSeviyesi)
6      {
7          if (BenzinSeviyesi > 10 || BataryaSeviyesi > 10)
8              return true;
9          else
10             return false;
11     }
12 }
13
14 class Ignition
15 {
16     static void Main(string[] args)
17     {
18         int BataryaSeviyesi = 95;
19         int BenzinSeviyesi = 80;
20         bool SuruseUygunluk = Araba.Calistir(BataryaSeviyesi, BenzinSeviyesi);
21         Console.WriteLine("Araba Calisti mi?:{0}", SuruseUygunluk);
22     }
23 }
```

Method Examples – Static Method

- With the word public; the compiler figures out that this method can be accessed from any class. If the word public had not been written, this method would only be accessible from the Ignition class.

```
1  using System;
2
3  class Araba
4  {   // Pay attention to public class.
5      public bool Calistir(int BataryaSeviyesi, int BenzinSeviyesi)
6      {
7          if (BenzinSeviyesi > 10 || BataryaSeviyesi > 10)
8              return true;
9          else
10             return false;
11     }
12 }
13
14 class Ignition
15 {
16     static void Main(string[] args)
17     {
18         int BataryaSeviyesi = 95;
19         int BenzinSeviyesi = 80;
20         Araba myCar = new Araba();
21         bool SuruseUygunluk = myCar.Calistir(BataryaSeviyesi, BenzinSeviyesi);
22         Console.WriteLine("Araba Calisti mi?: {0}", SuruseUygunluk);
23     }
24 }
```

Method Examples – Non-Static Method

Methods- Specs and Cautions

1. While naming the methods, the rules that must be followed in variable naming should be followed. Furthermore, you can't define a second Main() method within the program
2. If a method does not return a value, it must be defined as "void". If there is no input parameter, the parentheses of the method are left empty. Ex: `public void MerhabaDunyaYaz(){Console.Write('HW')}`
3. A method without a body statement but only a return could be defined. If the return keyword is used without a value, the program will terminate.

Methods- Specs and Cautions

Try This:

```
1  using System;
2
3  class YakitSistemi
4  {
5      static void UyariVer(string mesaj, int miktar)
6      {
7          if (miktar > 10) return;
8
9          for (; miktar > 0; miktar--)
10             Console.WriteLine($"{mesaj}! Kalan: {miktar}");
11     }
12
13     static void Main()
14     {
15         UyariVer("Dusuk Yakit", 5);
16     }
```

What is the message in the console?

Methods- Specs and Cautions

What is the problem here?

4. You can't use return statement in a method where return type is void.

Incorrect Use →

```
class Method // Gazi IE - Method Exercise
{
    1 başvuru
    static void operation(int x, int y)
    {
        return a+b;
    }
    0 başvuru
    static void Main(string[] args)
    {
        int z = operation(5, 4);
    }
}
```

Methods- Specs and Cautions

5. When returning a value from a method, the type of returned value should be carefully matched.
6. When passing parameters to methods, type conversion rules must be followed, and appropriate types must be used.
7. When getting a value from a return keyword of a method, make sure the returned value matches the the assigned value.

Ex: `int toplam = topla(a,b);`

```
class Method // Gazi IE - Method Exercise
{
    1 başvuru
    static void operation(int x, int y)
    {
        Console.WriteLine(x+y);
        return;
    }
    0 başvuru
    static void Main(string[] args)
    {
        double a, b;
        a = 1.23D;
        b = 5.43D;
        operation(a, b);
    }
}
```

Incorrect

Methods- Specs and Cautions

8. Number of parameters, the order of parameters and type of parameters must match to the definition of the method when invoking a method. Method calls with missing, mixed ordering and wrong parameter types are error-prone.

9. A method doesn't have to always accept parameters or return values, it might just simply do an operation without interception. Check out code below:

```
1  class Methods
2  {
3      static void Display()
4      {
5          Console.Write("Hello");
6      }
7
8      static void Main() {
9          Display();
10     }
11 }
```

Methods- Specs and Cautions

- A method cannot be declared inside another method.

Misuse →

```
class Program
{
    // GaziIE - Method Exercise
    1 başvuru
    static void operation(int a, int b)
    {
        int c = a + b;
        return;
        static int multipy(int x, int y)
        {
            return x * y;
        }
    }

    0 başvuru
    static void Main(string[] args)
    {
        int num1=7, num2=3;
        operation(num1, num2);
        multipy(num1,num2);
    }
}
```

Important Issues for Methods

10. The types of the variables in the method parameter brackets must be specified individually.

```
class Program
{
    // GaziIE - Method Exercise
    1 başvuru
    static int operation(int a, b, c)
    {
        int d = a + b + c;
        return d;
    }

    0 başvuru
    static void Main(string[] args)
    {
        int num1=7, num2=3, num3=5;
        operation(num1, num2, num3);

    }
}
```

Every variable needs its own type to be defined

Error

Methods- Specs and Cautions

11. Defining method parameters inside the method is also not allowed.

Incorrect

```
class Program
{
    // GaziIE - Method Exercise
    1 başvuru
    static int operation(int a, b)
    {
        int b;
        int c = a + b ;
        return c;
    }

    0 başvuru
    static void Main(string[] args
    {
        int num1=7, num2=3;
        operation(num1, num2);

    }
}
```

Methods- Specs and Cautions

12. Variables declared inside a method do not remain in memory for the entire duration of the program. They come into existence when the method begins execution and are typically removed when the method finishes execution.

- If the same method is called again later, those variables are created again and used during that method call and finally removed once the method ends.
- Such variables are called local variables, and their lifetime is limited to the execution of the method in which they are declared.

Method Exercise – Find the fastest car

- Try this:

```
using System;

class ArabaTesti
{
    static string HizKarsilastir(int hiz1, int hiz2)
    {
        if (hiz1 > hiz2)
            return "Tesla";
        return "TOG";
    }

    static void Main(string[] args)
    {
        int TeslaHiz, TogHiz;
        Console.Write("Please Enter current speed of Tesla:");
        TeslaHiz = int.Parse(Console.ReadLine());
        Console.Write("Please Enter current speed of TOG:");
        TogHiz = int.Parse(Console.ReadLine());

        string enHizli = HizKarsilastir(TeslaHiz, TogHiz);
        Console.WriteLine("Su an en hizli arac: {0}", enHizli);
    }
}
```

Method Exercise – Find the Greatest of 3

TRY THIS:

```
using System;

class ArabaTesti
{
    static string HizKarsilastir2(int h1, string n1, int h2, string n2)
    {
        if (h1 > h2)
            return n1;
        return n2;
    }

    static string HizKarsilastir3(int h1, string n1, int h2, string n2, int h3, string n3)
    {
        string araHizli = HizKarsilastir2(h2, n2, h3, n3);
        int hAra = (araHizli == n2) ? h2 : h3;

        return HizKarsilastir2(h1, n1, hAra, araHizli);
    }

    static void Main(string[] args)
    {
        int TeslaHiz, TogHiz, BydHiz;

        Console.Write("Please Enter current speed of Tesla:");
        TeslaHiz = int.Parse(Console.ReadLine());
        Console.Write("Please Enter current speed of TOGG:");
        TogHiz = int.Parse(Console.ReadLine());
        Console.Write("Please Enter current speed of BYD:");
        BydHiz = int.Parse(Console.ReadLine());

        string enHizli = HizKarsilastir3(TeslaHiz, "Tesla", TogHiz, "TOGG", BydHiz, "BYD");
        Console.WriteLine("Su an en hizli arac: {0}", enHizli);
    }
}
```

Method Parameters as Arrays

- Since arrays are treated as a separate type in C#, passing arrays to the methods is the same as passing normal data types but pay attention to square brackets [].

```
class Program
{
    // GaziIE - Method Exercise (Arrays)
    1 başvuru
    static void Display(int[] myarr)
    {
        foreach (var item in myarr)
        {
            Console.WriteLine(item);
        }
    }

    0 başvuru
    static void Main(string[] args)
    {
        int[] numbers = { 11, 55, 66, 12, 34, 78 };
        Display(numbers);
    }
}
```

Method Parameters as Arrays

- Let’s write a method that prints element of an array of all types, not just int[] :

```
class Program
{
    // GaziIE - Method Exercise (Arrays)
    1 başvuru
    static void Display(Array myarr)
    {
        foreach (object item in myarr)
        {
            Console.WriteLine(item);
        }
    }

    0 başvuru
    static void Main(string[] args)
    {
        object[] numbers = { 11, "Murat", 66, 5.45, 'c', -6 };
        Display(numbers);
    }
}
```

THE END

CHECK OUT COURSE WEBSITE: HTTPS://GAZI-END-MUH.GITHUB.IO

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM